22 de Abril 2026

DOCUMENTACIÓN

SISTEMA DE SIMULACIÓN DE TRÁFICO VEHICULAR

Emanuel De La Cruz Balbuena (2024-1720) **- Lider**

Wandrys Yasirys Ferrand Guzman (2024-2038)

Euriel Fernelis Mateo De Oleo (2024-1681)

Erick Leonardo Perez Veloz

Programación Paralela

Indice

1) INTRODUCCIÓN 4
Presentación General 4
Justificación del Tema 4
Objetivos del Proyecto 4
Objetivo General 4
Objetivos Específicos 5
2) Descripción del Problema 5
Contexto del problema seleccionado 5
Aplicación del problema en un escenario real 6
Importancia del paralelismo en la solución 6
3) Cumplimiento de los Requisitos del Proyecto 6
4) Diseño de la Solución 7
Arquitectura general del sistema 7
1. Ciclo de Desarrollo y Calidad (src y tests) 7
2. Gestión y Rendimiento (docs y metrics) 7
Diagrama de componentes/tareas paralelas 8
Estrategia de paralelización utilizada 8
Descomposición exploratoria 8
Herramientas y tecnologías empleadas (C#, TPL, etc.) 9
5) Implementación Técnica 9
Descripción de la estructura del proyecto: 9
1. Separación de Responsabilidades (App vs. Pruebas) 10
2. Organización por Capas (Modelado y Control) 10
Explicación del código clave 10
Uso de mecanismos de sincronización 11
Justificación técnica de las decisiones tomadas 11
6) Evaluación de Desempeño 12
Comparativa entre ejecución secuencial y paralela 12
Métricas: tiempo de ejecución, eficiencia, escalabilidad 13

Gráficas o tablas con resultados 13
Análisis de cuellos de botella o limitaciones 14
7) Trabajo en Equipo 14
8) Conclusiones 14
Principales aprendizajes técnicos 14
Retos enfrentados y superados 15
Posibles mejoras o líneas futuras 15
9) Referencias - Fuentes bibliográficas, técnicas o académicas consultadas 16
10) MANUAL DE EJECUCIÓN DEL SISTEMA Y DESCRIPCIÓN DEL FUNCIONAMIENTO. 17
Clase Semaforo 23
Vehículo 25
Conexión de todo en Program 26
Clase DetectorColisiones 34
Cambios en el Vehículo 35
Cambios en la Interfaz y el Programa principal 35

1) INTRODUCCIÓN

Presentación General

Este proyecto consiste en el desarrollo de un **Simulador de Tráfico Vehicular** de alto rendimiento, diseñado bajo el paradigma de la **Programación Paralela y Concurrencia** en C#. El sistema modela un entorno urbano donde múltiples vehículos operan como unidades de ejecución independientes (hilos/tareas).

A diferencia de una simulación secuencial, este sistema permite que cada vehículo tome decisiones en tiempo real basándose en el estado compartido de la infraestructura, como semáforos, intersecciones y la proximidad de otros vehículos, aprovechando al máximo la capacidad multi-núcleo de los procesadores modernos.

Justificación del Tema

La elección de este tema se fundamenta en la complejidad intrínseca de los sistemas de transporte modernos. La simulación de tráfico es un problema "naturalmente paralelo": en la vida real, los conductores no esperan a que otro termine su trayecto para moverse; todos actúan simultáneamente.

- **Relevancia Técnica:** Implementar esto requiere resolver desafíos críticos de la computación paralela, como la **sincronización de recursos compartidos** (evitar que dos vehículos ocupen el mismo espacio en memoria/vía) y la gestión de la latencia en la actualización de estados.
- **Valor Académico:** Permite aplicar conceptos avanzados de la **Task Parallel Library (TPL)** y el manejo de **colecciones concurrentes**, proporcionando un escenario tangible para medir el Speedup y la eficiencia del hardware.

Objetivos del Proyecto

Objetivo General

Desarrollar un simulador de tráfico vehicular capaz de ejecutar múltiples agentes de forma simultánea, utilizando técnicas de descomposición de tareas y datos para analizar el comportamiento del flujo vial y el rendimiento computacional del sistema bajo diferentes niveles de paralelismo.

Objetivos Específicos

- **Implementar la ejecución concurrente:** Lograr que cada vehículo funcione como una tarea independiente gestionada mediante Parallel.ForEach o Task.Run, integrando el control de MaxDegreeOfParallelism.

- **Gestionar la infraestructura compartida:** Diseñar un sistema de control para semáforos e intersecciones que utilice mecanismos de sincronización (como lock o SemaphoreSlim) para prevenir condiciones de carrera.

- **Evaluar la escalabilidad:** Realizar pruebas de carga aumentando el número de vehículos y vías para medir cómo responde el sistema ante un incremento de recursos de CPU.

- **Analizar métricas de rendimiento:** Calcular y presentar datos precisos sobre el tiempo de procesamiento, detectando el punto de saturación y la ganancia de velocidad (Speedup) en comparación con una ejecución de un solo hilo.

- **Visualizar resultados:** Ofrecer una interfaz (consola avanzada o gráfica) que reporte colisiones detectadas y el uso de recursos en tiempo real.

2) Descripción del Problema

Contexto del problema seleccionado

El tráfico vehicular urbano representa uno de los sistemas más complejos de modelar computacionalmente. En una ciudad, cientos o miles de vehículos se desplazan simultáneamente por una red de calles e intersecciones, cada uno tomando decisiones individuales que afectan el comportamiento colectivo del flujo. Simular este comportamiento de forma secuencial, es decir, procesando un vehículo a la vez, no refleja la realidad del sistema y genera tiempos de cómputo que escalan linealmente con la cantidad de agentes, haciéndose inviable a gran escala.

El problema central consiste en coordinar múltiples agentes concurrentes que comparten recursos críticos: las intersecciones, los semáforos y los carriles de circulación. Cuando dos vehículos intentan ocupar el mismo espacio al mismo tiempo, se produce una condición de carrera que en términos de software representa un error de sincronización, y en términos del modelo representa una colisión. El sistema debe detectar, prevenir y registrar estos eventos manteniendo el rendimiento bajo carga.

Aplicación del problema en un escenario real

Este tipo de simulación tiene aplicación directa en la planificación urbana y la ingeniería de tránsito. Los municipios y organismos de transporte utilizan simuladores para evaluar el impacto de cambios en la infraestructura vial, como la instalación de nuevos semáforos, la modificación de sentidos de circulación o la habilitación de carriles exclusivos, antes de ejecutar obras costosas en el mundo real.

También tiene aplicación en el desarrollo de sistemas de navegación tipo GPS, donde se necesita calcular en tiempo real las rutas óptimas considerando el estado actual del tráfico en múltiples puntos de la ciudad de forma simultánea. Empresas como Waze o Google Maps procesan miles de posiciones de vehículos en paralelo para actualizar sus modelos de congestión.

Importancia del paralelismo en la solución

La naturaleza del problema es inherentemente paralela: en la realidad, los vehículos no esperan turno para moverse. Implementar la solución con un enfoque secuencial supondría simular los vehículos uno a uno, perdiendo por completo la interacción simultánea que define el comportamiento del tráfico real.

El uso de Parallel.ForEach con MaxDegreeOfParallelism permite que el sistema escale de acuerdo con los recursos disponibles en el hardware. A mayor número de núcleos asignados, mayor cantidad de vehículos pueden procesarse en el mismo intervalo de tiempo, reduciendo el tiempo total de simulación y permitiendo modelar escenarios urbanos de mayor complejidad. El paralelismo no es una optimización opcional en este problema, es un requisito fundamental para que la simulación sea representativa de la realidad.

3) Cumplimiento de los Requisitos del Proyecto

1. **Ejecución simultánea de múltiples tareas:** Cada vehículo es un hilo o tarea que se ejecuta de forma concurrente.

2. **Compartición de datos:** Se comparten datos como:
   - Estado de semáforos
   - Posiciones en la vía
   - Intersecciones

3. **Descomposición exploratoria:** Se puede dividir por:
   - Vehículos
   - Zonas de la ciudad
   - Tipos de comportamiento

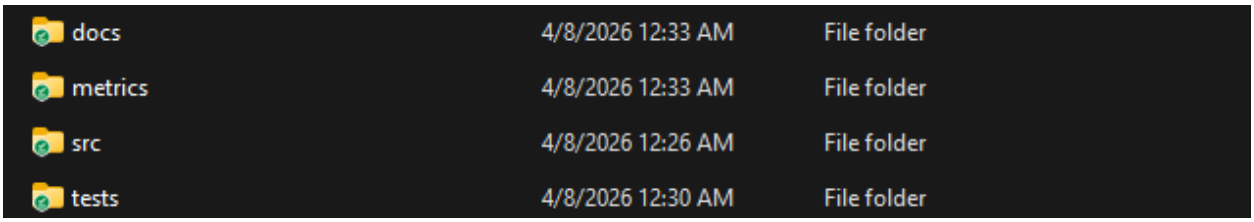
4. **Escalabilidad:** Se pueden agregar más vehículos o calles y el sistema seguirá funcionando con más recursos. Se utilizarán procesadores indicados por el usuario desde el inicio con MaxDegreeOfParallelism.

5. **Métricas de rendimiento:**
   - Tiempo de simulación
   - Cantidad de vehículos procesados
   - Colisiones detectadas
   - Uso de CPU • Speedup: Tiempo secuencial / Tiempo paralelo
   - Eficiencia: Speedup / Número de Procesadores

6. **Escenario real:** Aplicable a planificación urbana, apps de navegación tipo GPS y estudios de tráfico.

4) Diseño de la Solución

Arquitectura general del sistema



docs	4/8/2026 12:33 AM	File folder
metrics	4/8/2026 12:33 AM	File folder
src	4/8/2026 12:26 AM	File folder
tests	4/8/2026 12:30 AM	File folder

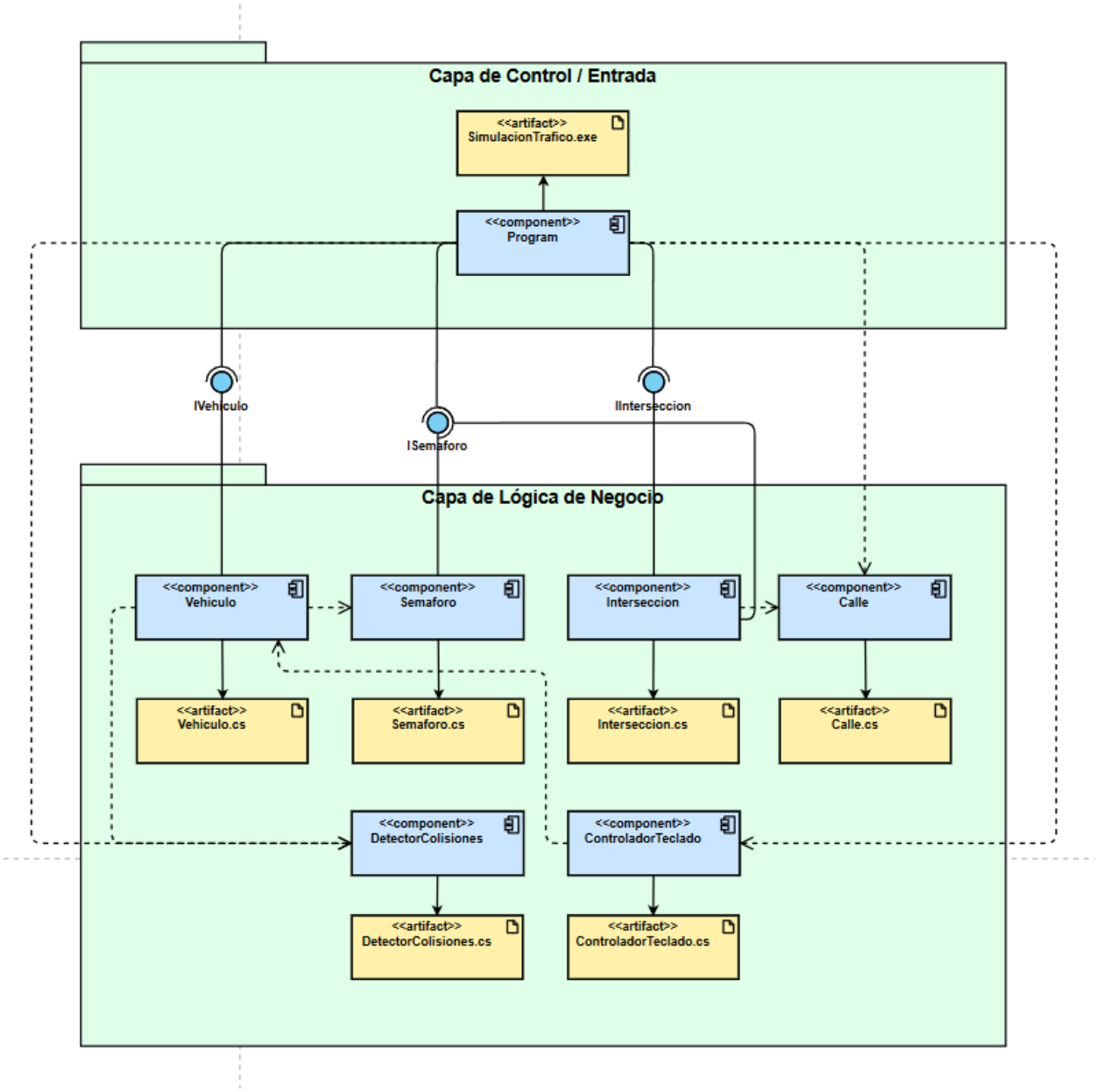
1. Ciclo de Desarrollo y Calidad (src y tests)

- **src**: Es la carpeta de código fuente (source code) donde se encuentra toda la lógica, algoritmos y funcionalidades principales del proyecto.
- **tests**: Contiene los archivos de pruebas encargados de validar que el código en src funcione correctamente y no presente fallos ante cambios futuros.

2. Gestión y Rendimiento (docs y metrics)

- **docs**: Repositorio dedicado a la documentación técnica, manuales de usuario y diagramas que explican el diseño del sistema.
- **metrics**: Espacio para almacenar reportes de rendimiento y estadísticas de ejecución, lo cual permite analizar la eficiencia y velocidad del software bajo diferentes condiciones.

Diagrama de componentes/tareas paralelas



Estrategia de paralelización utilizada

Descomposición exploratoria

Cada vehículo representa una unidad de trabajo independiente, lo que permite descomponerlos naturalmente en tareas paralelas. No existe un orden obligatorio entre vehículos, por lo que cada

uno puede simular su recorrido completo de forma autónoma sin depender del progreso de los demás. Los recursos compartidos como el semáforo y el detector de colisiones actúan como puntos de sincronización controlada entre los hilos.

Herramientas y tecnologías empleadas (C#, TPL, etc.)

- **Lenguaje y plataforma:** C# con .NET 8, usando programación orientada a objetos e interfaces para estructurar el sistema.

- **Task Parallel Library (TPL):** Parallel.ForEach para ejecutar vehículos concurrentemente y Task.Run para el ciclo automático del semáforo.

- **Sincronización:** lock para proteger estructuras compartidas, Interlocked para fcontadores atómicos y SemaphoreSlim para control de acceso thread-safe.

- **Medición:** Stopwatch para tiempos de ejecución y Process.TotalProcessorTime para uso de CPU.

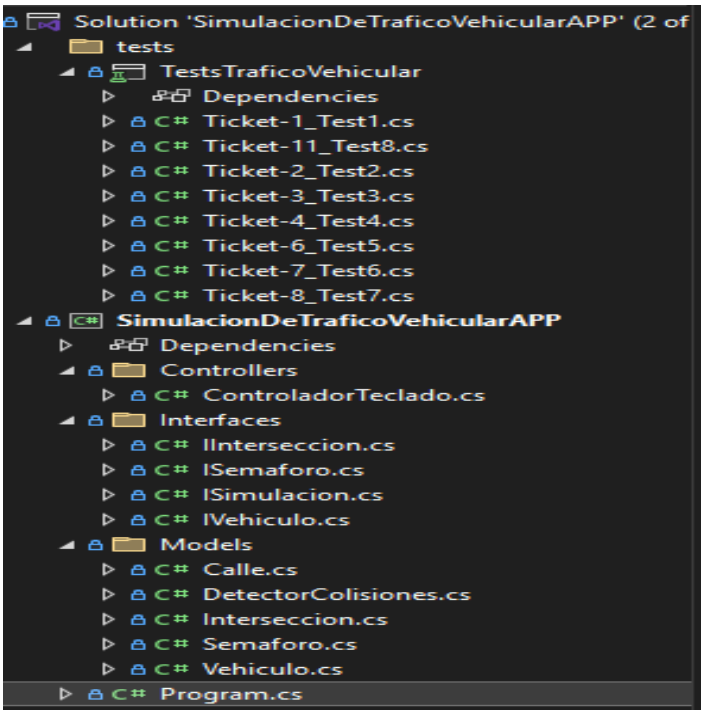
- **Cancelación:** CancellationTokenSource para detener la simulación de forma controlada.

- **Testing:** xUnit para pruebas unitarias de concurrencia y comportamiento thread-safe.

5) Implementación Técnica

Descripción de la estructura del proyecto:

SRC: ESTRUCTURA



1. Separación de Responsabilidades (App vs. Pruebas)

El sistema está dividido en dos proyectos independientes para mantener el orden y la calidad:

- **SimulacionDeTraficoVehicularAPP:** Es el núcleo del sistema. Aquí reside toda la lógica operativa, los objetos y la ejecución principal del simulador.
- **TestsTraficoVehicular:** Es un entorno de validación. Contiene pruebas unitarias (organizadas por "Tickets") que verifican que cada parte del código funcione correctamente antes de integrarla, lo que evita errores en cadena.

2. Organización por Capas (Modelado y Control)

Dentro de la aplicación principal, el código se agrupa según su función específica:

- **Interfaces y Models:** Es la base del sistema. Las **Interfaces** definen los "contratos" o reglas (qué debe hacer un vehículo o un semáforo), mientras que los **Models** son las clases físicas (Calle, Vehículo, Detector de Colisiones) que contienen la lógica de datos y comportamiento.

- **Controllers y Program:** Es la capa de gestión. El **Controlador** actúa como puente para capturar las entradas del usuario (teclado), y **Program.cs** funciona como el punto de entrada que orquestra el inicio de toda la simulación.

Explicación del código clave

Ciclo paralelo principal

El corazón de la simulación es el Parallel.ForEach que toma la lista de vehículos pendientes y los ejecuta simultáneamente en múltiples hilos. Cada vehículo corre su recorrido completo de forma

independiente, pasando por semáforos, intersecciones y detección de colisiones sin bloquear a los demás.

Semáforo con ciclo automático

La clase Semaforo lanza un Task.Run en segundo plano que cambia su estado Verde → Amarillo → Rojo de forma continua. El estado se lee y escribe siempre dentro de un lock para que múltiples hilos puedan consultarlo sin corromperse entre sí.

Detección de colisiones

DetectorColisiones mantiene un diccionario de posiciones activas. Cada vez que un vehículo se mueve registra su nueva posición y si encuentra otro vehículo en ese lugar se detecta la colisión, se incrementa el contador y ambos quedan marcados como eliminados.

Uso de mecanismos de sincronización

lock protege el diccionario de posiciones en DetectorColisiones y el estado del semáforo, garantizando que solo un hilo pueda modificarlos a la vez.

Interlocked.Increment actualiza el contador de colisiones y el de vehículos completados de forma atómica, sin necesidad de un lock completo ya que son operaciones sobre un solo entero.

SemaphoreSlim está declarado en Semaforo para controlar el acceso concurrente a la intersección, permitiendo extender fácilmente cuántos vehículos pueden pasar simultáneamente.

CancellationToken se propaga a cada vehículo durante su simulación, permitiendo que todos los hilos se detengan limpiamente cuando el usuario cancela o la simulación termina sola.

Justificación técnica de las decisiones tomadas

Parallel.ForEach sobre Task manual: Se eligió Parallel.ForEach porque los vehículos son unidades de trabajo homogéneas y de duración similar, lo que permite que el TPL gestione automáticamente la distribución de hilos sin necesidad de administrar tareas individualmente.

lock sobre otros mecanismos en estructuras compartidas: El diccionario de posiciones y el estado del semáforo requieren operaciones de lectura y escritura compuestas que no pueden garantizarse con Interlocked solo. El lock garantiza atomicidad en bloques de múltiples instrucciones.

Interlocked para contadores: Los contadores de colisiones y vehículos completados son incrementos simples sobre un entero, donde Interlocked.Increment es más eficiente que un lock completo ya que opera a nivel de instrucción de CPU sin bloquear el hilo.

CancellationTokenSource para control de ciclo de vida: En lugar de usar flags booleanos volátiles, se optó por CancellationToken porque es el mecanismo estándar de .NET para cancelación cooperativa, compatible nativamente con Parallel.ForEach y Task.

Semilla fija en Random: Se fijó la semilla del generador aleatorio para que ambas versiones, secuencial y paralela, tengan exactamente los mismos tiempos de espera, haciendo la comparación de Speedup válida y reproducible.

ThreadLocal<Random>: Se usa una instancia de Random por hilo para evitar condiciones de carrera, ya que Random no es thread-safe y compartir una sola instancia entre hilos genera resultados incorrectos o excepciones.

6) Evaluación de Desempeño

Comparativa entre ejecución secuencial y paralela

La comparación entre la ejecución secuencial y la ejecución paralela del sistema permite evidenciar las ventajas del uso de múltiples procesadores en el procesamiento de los vehículos dentro de la simulación. En la ejecución secuencial, las tareas se realizan de manera ordenada, procesando un vehículo a la vez, lo que provoca un mayor tiempo total de ejecución debido a la acumulación de esperas y operaciones.

Por otro lado, en la ejecución paralela, los vehículos son procesados de manera concurrente, distribuyendo la carga de trabajo entre varios procesadores. Esto permite que múltiples vehículos se simulen al mismo tiempo, reduciendo significativamente el tiempo total de ejecución.

Los resultados obtenidos demuestran que la ejecución paralela supera ampliamente a la secuencial en términos de rendimiento. Por ejemplo, con 8 procesadores, el tiempo secuencial fue de 176.06 segundos, mientras que el tiempo paralelo se redujo a solo 10.03 segundos, evidenciando una mejora considerable. De igual forma, en configuraciones con 2 y 4 procesadores también se observa una reducción progresiva del tiempo paralelo en comparación con el secuencial.

Además, el uso de paralelismo permite un mejor aprovechamiento de los recursos del sistema, incrementando el speedup y la eficiencia, lo que contribuye a una ejecución más rápida y efectiva. Sin embargo, es importante considerar que la ejecución paralela puede implicar cierta sobrecarga por la gestión de hilos y sincronización, aunque en este caso los beneficios superan ampliamente dichas limitaciones.

Métricas: tiempo de ejecución, eficiencia, escalabilidad

Tiempo de Ejecución: El tiempo de ejecución se evaluó comparando la versión secuencial y la versión paralela del sistema utilizando diferentes cantidades de procesadores. Los resultados obtenidos evidencian una disminución significativa en el tiempo de ejecución paralelo a medida que se incrementa el número de procesadores. En la prueba con 2 procesadores, el tiempo secuencial fue de 283.73 segundos y el paralelo de 78.35 segundos. Con 4 procesadores, el tiempo secuencial fue de 199.38 segundos y el paralelo de 16.98 segundos. Finalmente, con 8 procesadores, el tiempo secuencial fue de 176.06 segundos y el paralelo de 10.03 segundos. Esto demuestra que el uso de paralelismo mejora considerablemente el rendimiento del sistema.

Eficiencia: La eficiencia permite medir el nivel de aprovechamiento de los procesadores durante la ejecución paralela. En este caso, se obtuvieron valores de eficiencia de 181.06% con 2 procesadores, 293.56% con 4 procesadores y 219.40% con 8 procesadores. Estos valores superiores al 100% pueden deberse a diferencias en la carga de trabajo entre las versiones secuencial y paralela, así como a la reducción de tiempos de espera en la ejecución concurrente, lo que favorece el rendimiento general del sistema.

Escalabilidad: La escalabilidad del sistema se analizó mediante el speedup obtenido en cada una de las pruebas. Con 2 procesadores se alcanzó un speedup de 3.62x, con 4 procesadores un speedup de 11.74x y con 8 procesadores un speedup de 17.55x. Estos resultados indican que el sistema mejora su rendimiento a medida que se incrementa el número de procesadores, lo que evidencia una buena capacidad de escalabilidad y un uso eficiente del procesamiento paralelo.

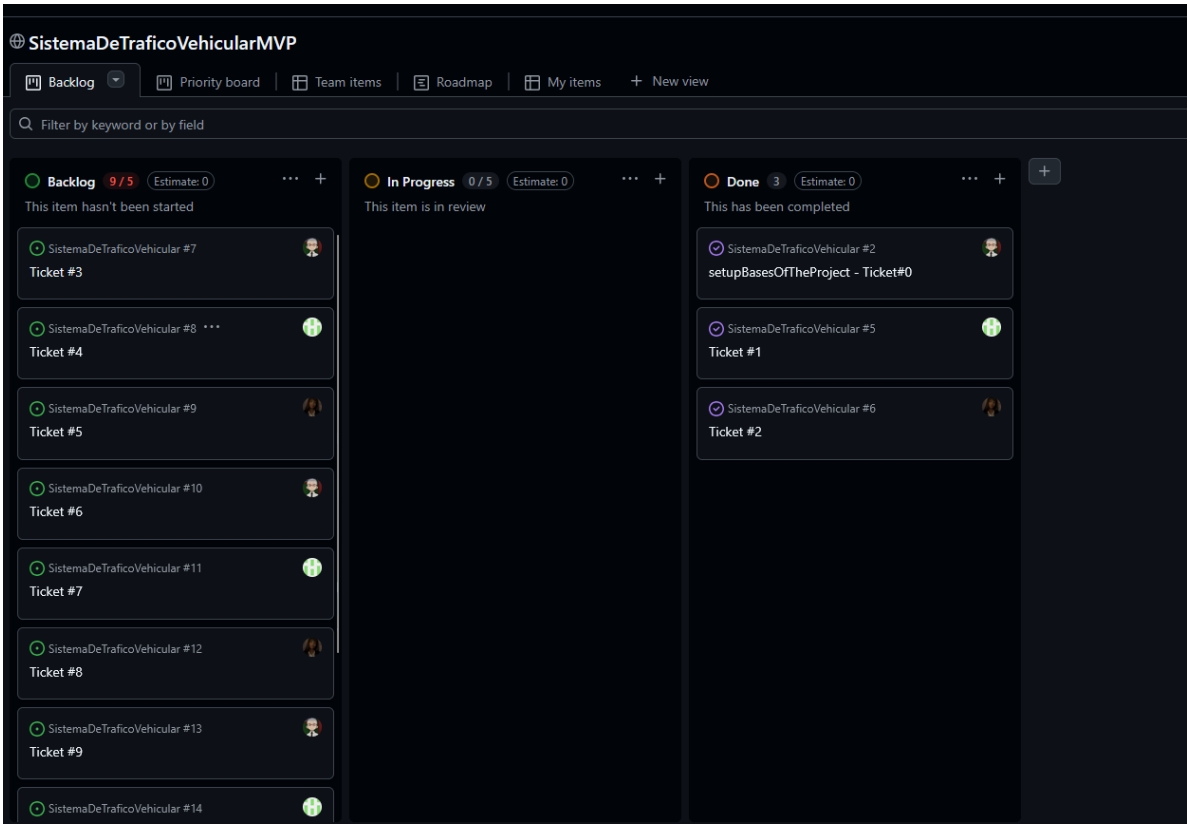
Gráficas o tablas con resultados

Procesadores usados	**2**	**4**	**8**
Vehiculos completados	25	11	14
Colisiones detectadas	20	11	10
Tiempo secuencial	283.73s	199.38s	176.06s
Tiempo paralelo	78.35s	16.98s	10.03s
CPU usada	453ms	94 ms	62ms
Speedun	3.62x	11.74x	17.55x
Eficiencia	181.06%	293.56%	219.40%

Análisis de cuellos de botella o limitaciones

7) Trabajo en Equipo

- **Descripción del reparto de tareas:** Utilizamos un tablero **kanban** que nos ofrece Github aplicando tickets para el reparto de tareas para el grupo en el cual el lider le asignó 11 tickets a los integrantes para abarcar el proceso completo del proyecto.



- **Herramientas utilizadas para coordinación: Github** (para el código y reparto de tareas), **Microsoft Teams** (para las reuniones) y **Microsoft Word** para la edición de documentos.

8) Conclusiones

Principales aprendizajes técnicos

Trabajar con Parallel.ForEach enseñó que el paralelismo no es gratuito: requiere identificar correctamente qué recursos son compartidos y protegerlos. Se aprendió que lock e Interlocked no

son intercambiables, cada uno tiene su caso de uso según la complejidad de la operación. También quedó claro que medir rendimiento correctamente exige condiciones idénticas entre versiones, de ahí la semilla fija en Random. Finalmente se comprendió que una eficiencia superior al 100% no siempre es un error lógico sino una señal de medición incorrecta.

Retos enfrentados y superados

El mayor reto fue garantizar que el estado del semáforo fuera consistente entre múltiples hilos sin generar deadlocks. Se resolvió usando un lock dedicado solo para el estado, separado de cualquier otra operación. Otro reto fue la detección de colisiones dobles, donde un vehículo ya eliminado seguía corriendo y chocando nuevamente. Se resolvió con un HashSet de vehículos eliminados consultado al inicio de cada iteración. El cálculo de Speedup también fue un desafío porque el tiempo secuencial ficticio producía valores matemáticamente incorrectos, resuelto ejecutando una versión secuencial real como baseline.

Posibles mejoras o líneas futuras

Se podría reemplazar la espera activa con Thread.Sleep(100) en los semáforos por un mecanismo basado en eventos como ManualResetEventSlim, reduciendo el consumo de CPU durante las esperas. La simulación podría extenderse a múltiples intersecciones con semáforos independientes para aumentar la complejidad del escenario. También sería valioso implementar un sistema de métricas en tiempo real que muestre el progreso durante la ejecución y no solo al finalizar. Finalmente, aumentar la cobertura de pruebas unitarias con escenarios de stress con cientos de vehículos simultáneos daría mayor confianza en la robustez del sistema bajo carga real.

9) **Referencias -** Fuentes bibliográficas, técnicas o académicas consultadas

https://learn.microsoft.com/en-us/dotnet/api/system.threading.interlocked?view=net-10.0

https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/statements/lock

https://medium.com/@wgyxxbf/thread-security-in-c-547e5f7cfe2b

https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/how-to-write-a-simple-parallel-foreach-loop

https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.parallelooptions.maxdegreeofparallelism?view=net-10.0

https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtoken?view=net-10.0

10) MANUAL DE EJECUCIÓN DEL SISTEMA Y DESCRIPCIÓN DEL FUNCIONAMIENTO.

Ticket #1 — Configuración base del proyecto y estructura de namespaces

Descripción: Se estableció la estructura base del proyecto de simulación de tráfico vehicular en C#, definiendo las interfaces principales y el punto de entrada de la aplicación.

Interfaces definidas

IVehiculo

Contrato base para todos los vehículos que participan en la simulación.

```
namespace SimulacionDeTraficoVehicularAPP.Interfaces
{
    4 referencias
    public interface IVehiculo
    {
        0 referencias
        int Id { get; }
        0 referencias
        string Tipo { get; }          // "Auto", "Bus", "Moto"
        0 referencias
        int VelocidadActual { get; set; }
        0 referencias
        (int X, int Y) Posicion { get; set; }
        0 referencias
        void Mover();
        0 referencias
        void Detener();
    }
}
```

ISemaforo

Contrato para los semáforos de la ciudad. Incluye el enum EstadoSemaforo.

```
namespace SimulacionDeTraficoVehicularAPP.Interfaces
{
    6 referencias
    public enum EstadoSemaforo { Verde, Amarillo, Rojo }

    2 referencias
    public interface ISemaforo
    {
        0 referencias
        int Id { get; }
        0 referencias
        EstadoSemaforo Estado { get; }
        0 referencias
        void CambiarEstado(EstadoSemaforo nuevoEstado);
        0 referencias
        bool PuedeAvanzar();
    }
}
```

Interseccion

Contrato para las intersecciones entre calles. Gestiona semáforos y detecta colisiones.

```
1   namespace SimulacionDeTraficoVehicularAPP.Interfaces
2   {
        1 referencia
3       public interface IInterseccion
4       {
            0 referencias
5           int Id { get; }
            0 referencias
6           (int X, int Y) Coordenadas { get; }
            0 referencias
7           ISemaforo Semaforo { get; }
            0 referencias
8           bool HayColision(IVehiculo vehiculo);
            0 referencias
9           void RegistrarVehiculo(IVehiculo vehiculo);
            0 referencias
10          void LiberarVehiculo(IVehiculo vehiculo);
11      }
12  }
```

ISimulacion

Contrato principal que orquesta toda la simulación.

- Arranca la simulación con el número de procesadores indicado
- Detiene la simulación en curso
- Muestra los resultados y métricas de rendimiento

```
1   namespace SimulacionDeTraficoVehicularAPP.Interfaces
2   {
        1 referencia
3       public interface ISimulacion
4       {
            0 referencias
5           void Iniciar(int maxProcesadores);
            0 referencias
6           void Detener();
            0 referencias
7           void MostrarMetricas();
8       }
9   }
```

Configuración de Program.cs

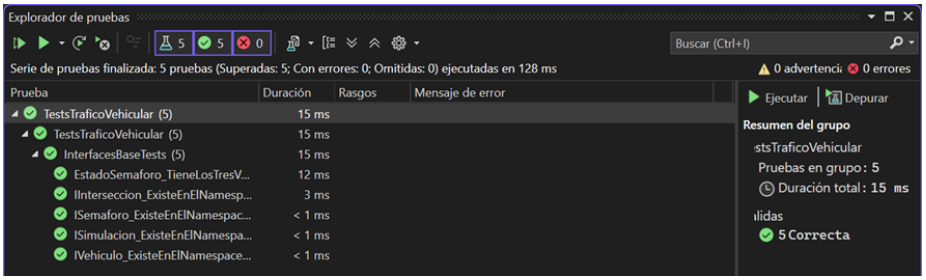
Fue configurado para cumplir con el criterio de aceptación de solicitar y validar el número de procesadores antes de iniciar cualquier simulación:

- Llama al método SolicitarProcesadores() que usa un bucle while hasta recibir entrada válida.
- El límite superior se obtiene dinámicamente con Environment.ProcessorCount.
- Rechaza valores no numéricos, menores a 1 o mayores al límite del sistema.
- Configura ParallelOptions con MaxDegreeOfParallelism listo para la siguiente tarea.

```
1   using SimulacionDeTraficoVehicularAPP.Interfaces;
2
3   namespace SimulacionDeTraficoVehicularAPP
4   {
        0 referencias
5       internal class Program
6       {
            0 referencias
7           private static void Main(string[] args)
8           {
9               Console.WriteLine(" Simulador de Tráfico Vehicular \n");
10
11              int maxProcesadores = SolicitarProcesadores();
12
13              var opciones = new ParallelOptions
14              {
15                  MaxDegreeOfParallelism = maxProcesadores
16              };
17
18              Console.WriteLine($"\nConfiguración lista: {maxProcesadores} procesadores asignados.");
19              Console.WriteLine("Interfaces definidas: IVehiculo, ISemaforo, IInterseccion, ISimulacion.");
20              Console.WriteLine("Proyecto listo para la siguiente tarea.");
21          }
22
            1 referencia
23          private static int SolicitarProcesadores()
24          {
25              int limite = Environment.ProcessorCount;
26
27              while (true)
28              {
29                  Console.Write($"Ingrese número de procesadores a usar (1 - {limite}): ");
30                  string? entrada = Console.ReadLine();
31
32                  if (int.TryParse(entrada, out int valor) && valor >= 1 && valor <= limite)
33                      return valor;
34
35                  Console.WriteLine($"  Valor inválido. Ingrese un número entre 1 y {limite}.");
36              }
37          }
38      }
39  }
```

Tests unitarios

Se crearon 5 pruebas unitarias. Todas pasaron exitosamente.



Ticket #2 – Simulacion de Trafico Vehicular

Descripcion: Este modulo implementa una simulacion de trafico vehicular utilizando concurrencia. Cada vehiculo actua de manera independiente, tomando decisiones dinamicas de movimiento o detencion en funcion de condiciones simuladas como semaforos y trafico.

Clase Vehiculo

La clase Vehiculo representa una entidad dentro de la simulacion. Implementa la interfaz IVehiculo y contiene la logica principal de comportamiento.

Propiedades:

- Id: Identificador unico del vehiculo
- Tipo: Tipo de vehiculo (Auto, Bus, Moto, etc.)
- VelocidadActual: Velocidad de desplazamiento
- Posicion: Coordenadas actuales (X, Y)

```
9 references
public class Vehiculo : IVehiculo
{
    private static readonly object consoleLock = new object();

    private static readonly ThreadLocal<Random> random =
        new ThreadLocal<Random>(() => new Random());

    7 references
    public int Id { get; }
    7 references
    public string Tipo { get; }
    4 references
    public int VelocidadActual { get; set; }
    20 references
    public (int X, int Y) Posicion { get; set; }

    7 references
    public Vehiculo(int id, string tipo)
    {
        Id = id;
        Tipo = tipo;
        VelocidadActual = random.Value.Next(1, 5);
        Posicion = (0, 0);
    }
```

Metodos:

- Mover(): Avanza el vehiculo sin especificar razón
- Detener(): Detiene el vehiculo sin especificar razón
- Mover(string razon): Avanza el vehiculo indicando la razón
- Detener(string razon): Detiene el vehiculo indicando la razón
- Simular(): Ejecuta el ciclo de vida del vehiculo

```
1 reference
public void Mover()
{
    Posicion = (Posicion.X + VelocidadActual, Posicion.Y);

    lock (consoleLock)
    {
        Console.WriteLine($"[Vehiculo {Id} - {Tipo}] Avanza a ({Posicion.X}, {Posicion.Y}) - Sin razon especificada.");
    }
}

1 reference
public void Detener()
{
    lock (consoleLock)
    {
        Console.WriteLine($"[Vehiculo {Id} - {Tipo}] Se detiene en ({Posicion.X}, {Posicion.Y}) - Sin razon especificada.");
    }
}

2 references
public void Mover(string razon)
{
    Posicion = (Posicion.X + VelocidadActual, Posicion.Y);

    lock (consoleLock)
    {
        Console.WriteLine($"[Vehiculo {Id} - {Tipo}] Avanza a ({Posicion.X}, {Posicion.Y}) - {razon}");
    }
}

3 references
public void Detener(string razon)
{
    lock (consoleLock)
    {
        Console.WriteLine($"[Vehiculo {Id} - {Tipo}] Se detiene en ({Posicion.X}, {Posicion.Y}) - {razon}");
    }
}
```

Logica de Simulacion:

- Durante la simulacion:
- Cada vehiculo se ejecuta en un hilo independiente
- Se generan decisiones aleatorias para simular:
  1. Semaforo en rojo
  2. Trafico o decision del conductor
  3. Via libre
  4. En cada iteracion se ejecuta una sola accion: mover o detener

```
2 references
public void Simular()
{
    for (int i = 0; i < 10; i++)
    {
        Thread.Sleep(random.Value.Next(200, 500));

        bool semaforoEnRojo = random.Value.Next(0, 10) < 3;
        bool decideDetenerse = random.Value.Next(0, 10) < 2;

        if (semaforoEnRojo)
        {
            Detener("Semaforo en rojo");
        }
        else if (decideDetenerse)
        {
            Detener("Trafico o decision del conductor");
        }
        else
        {
            Mover("Via libre");
        }
    }

    lock (consoleLock)
    {
        Console.WriteLine($"[Vehiculo {Id} - {Tipo}] Llego a su destino.\n");
    }
}
}
```

Concurrencia

- Se utiliza ThreadLocal<Random> para evitar conflictos entre hilos
- Se utiliza lock para sincronizar la salida en consola

Pruebas Unitarias

Se implementaron pruebas unitarias para validar:

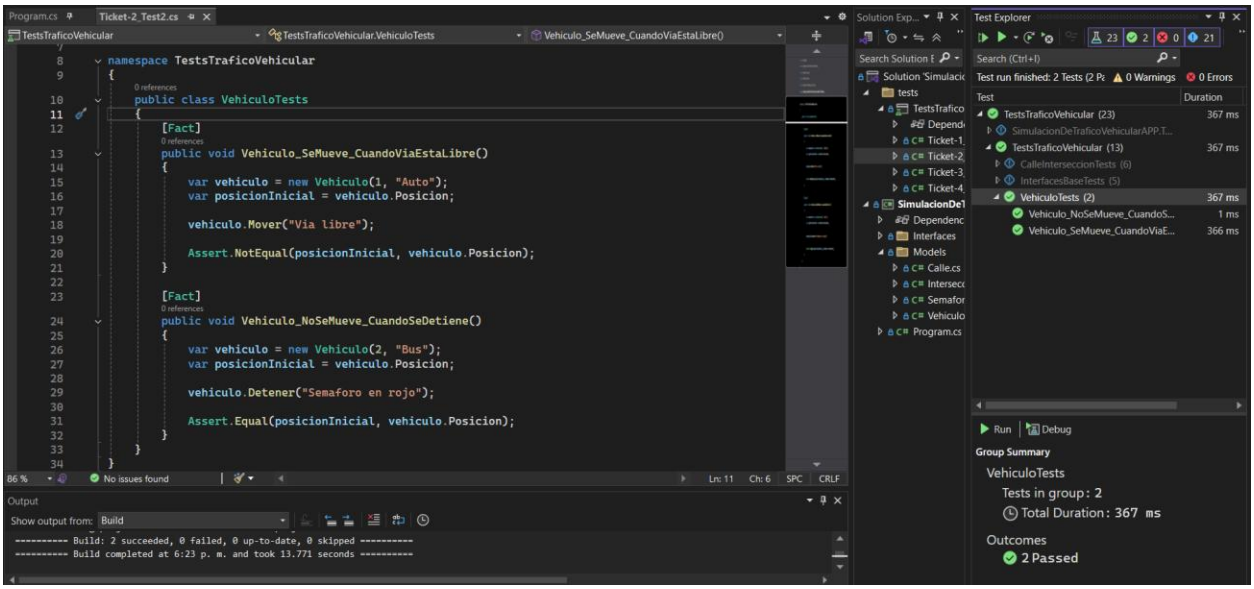
- Que el vehiculo se mueve correctamente cuando la via esta libre
- Que el vehiculo no cambia de posicion cuando se detiene

```
8   namespace TestsTraficoVehicular
9   {
        0 references
10      public class VehiculoTests
11      {
12          [Fact]
            0 references
13          public void Vehiculo_SeMueve_CuandoViaEstaLibre()
14          {
15              var vehiculo = new Vehiculo(1, "Auto");
16              var posicionInicial = vehiculo.Posicion;
17
18              vehiculo.Mover("Via libre");
19
20              Assert.NotEqual(posicionInicial, vehiculo.Posicion);
21          }
22
23          [Fact]
            0 references
24          public void Vehiculo_NoSeMueve_CuandoSeDetiene()
25          {
26              var vehiculo = new Vehiculo(2, "Bus");
27              var posicionInicial = vehiculo.Posicion;
28
29              vehiculo.Detener("Semaforo en rojo");
30
31              Assert.Equal(posicionInicial, vehiculo.Posicion);
32          }
33      }
```

Consideraciones: La logica de semaforo actualmente es simulada con valores aleatorios.

Test Unitario

Se implementaron pruebas unitarias utilizando para validar el comportamiento de la clase Vehiculo, específicamente los métodos Mover y Detener. Se verificó que el vehículo cambia de posición cuando la vía está libre y que mantiene su posición cuando se detiene, por ejemplo ante un semáforo en rojo. Las pruebas fueron ejecutadas mediante Test Explorer en Visual Studio y con el comando dotnet test, obteniendo resultados exitosos (2 pruebas pasadas y 0 fallidas), lo que confirma el correcto funcionamiento de la lógica implementada.



Ticket #3 – Implementar semáforos compartidos con SemaphoreSlim

Clase Semaforo

Esta es la pieza central. Se creó desde cero implementando la interfaz ISemaforo con tres responsabilidades principales:

Control de estado thread-safe: El estado del semáforo (Verde, Amarillo, Rojo) no se puede leer y escribir al mismo tiempo desde hilos distintos porque eso generaría condiciones de carrera. Para evitarlo, se protegió el acceso al estado con un candado interno, de modo que cuando un hilo lo está leyendo o modificando, los demás esperan su turno antes de acceder.

Ciclo automático sin deadlocks: El semáforo cambia solo de estado en segundo plano siguiendo el ciclo Verde → Amarillo → Rojo de forma continua. Se diseñó de manera que siempre avanza y nunca se queda esperando algo que no va a llegar, lo que garantiza que no haya deadlocks. Además, al terminar la simulación el ciclo se puede detener limpiamente sin dejar hilos colgados.

Tiempos configurables: La duración de cada estado se puede ajustar al crear el semáforo, sin necesidad de tocar la lógica interna.

```
namespace SimulacionDeTraficoVehicularAPP.Models
{
    14 references
    public class Semaforo : ISemaforo
    {
        private readonly SemaphoreSlim _semaphoreSlim;
        private readonly object _estadoLock = new object();
        private EstadoSemaforo _estado;
        private CancellationTokenSource _cts;

        // Tiempos configurables en milisegundos
        private readonly int _tiempoVerde;
        private readonly int _tiempoAmarillo;
        private readonly int _tiempoRojo;

        2 references
        public int Id { get; }

        9 references | 7/7 passing
        public EstadoSemaforo Estado
        {
            get { lock (_estadoLock) { return _estado; } }
        }

        11 references | 10/10 passing
        public Semaforo(int id, int tiempoVerde = 3000, int tiempoAmarillo = 1000, int tiempoRojo = 3000)
        {
            Id = id;
            _tiempoVerde = tiempoVerde;
            _tiempoAmarillo = tiempoAmarillo;
            _tiempoRojo = tiempoRojo;

            // Inicia en verde, permite múltiples hilos pasar simultáneamente
            _estado = EstadoSemaforo.Verde;
            _semaphoreSlim = new SemaphoreSlim(1, 1);
            _cts = new CancellationTokenSource();

            IniciarCiclo();
        }
```

```
9 references | 4/4 passing
public void CambiarEstado(EstadoSemaforo nuevoEstado)
{
    lock (_estadoLock)
    {
        _estado = nuevoEstado;
    }
}

8 references | 5/5 passing
public bool PuedeAvanzar()
{
    lock (_estadoLock)
    {
        return _estado == EstadoSemaforo.Verde;
    }
}

// Bloquea el hilo si el semáforo está en rojo, y lo libera cuando cambia a verde
0 references
public async Task EsperarSiEsNecesarioAsync(CancellationToken token = default)
{
    while (true)
    {
        lock (_estadoLock)
        {
            if (_estado == EstadoSemaforo.Verde)
                return; // puede avanzar
        }
        // Espera un momento y vuelve a revisar (polling liviano)
        await Task.Delay(100, token);
    }
}
```

```
6 references | 5/5 passing
public void Detener()
{
    _cts.Cancel();
}

1 reference
private void IniciarCiclo()
{
    Task.Run(async () =>
    {
        while (!_cts.Token.IsCancellationRequested)
        {
            // --- VERDE ---
            CambiarEstado(EstadoSemaforo.Verde);
            await Task.Delay(_tiempoVerde, _cts.Token).ContinueWith(_ => { });

            if (_cts.Token.IsCancellationRequested) break;

            // --- AMARILLO ---
            CambiarEstado(EstadoSemaforo.Amarillo);
            await Task.Delay(_tiempoAmarillo, _cts.Token).ContinueWith(_ => { });

            if (_cts.Token.IsCancellationRequested) break;

            // --- ROJO ---
            CambiarEstado(EstadoSemaforo.Rojo);
            await Task.Delay(_tiempoRojo, _cts.Token).ContinueWith(_ => { });
        }
    }, _cts.Token);
}
}
```

Se agregó void Simular(Semaforo semaforo) a la interfaz para mantener el contrato formal. En Program.cs se instancia **un único semáforo compartido** que se pasa a todos los vehículos.

```
namespace SimulacionDeTraficoVehicularAPP.Interfaces
{
    5 references
    public interface IVehiculo
    {
        7 references
        int Id { get; }
        7 references
        string Tipo { get; }          // "Auto", "Bus", "Moto"
        4 references
        int VelocidadActual { get; set; }
        20 references | 0/2 passing
        (int X, int Y) Posicion { get; set; }
        1 reference
        void Mover();
        1 reference
        void Detener();
        2 references
        void Simular(Semaforo semaforo);
    }
}
```

Vehículo

Antes cada vehículo decidía aleatoriamente si había un semáforo en rojo, lo cual era completamente ficticio. Ahora recibe el semáforo real y lo consulta en cada paso. Si está en rojo

o amarillo, el vehículo se detiene y espera pacientemente hasta que cambie a verde para continuar.

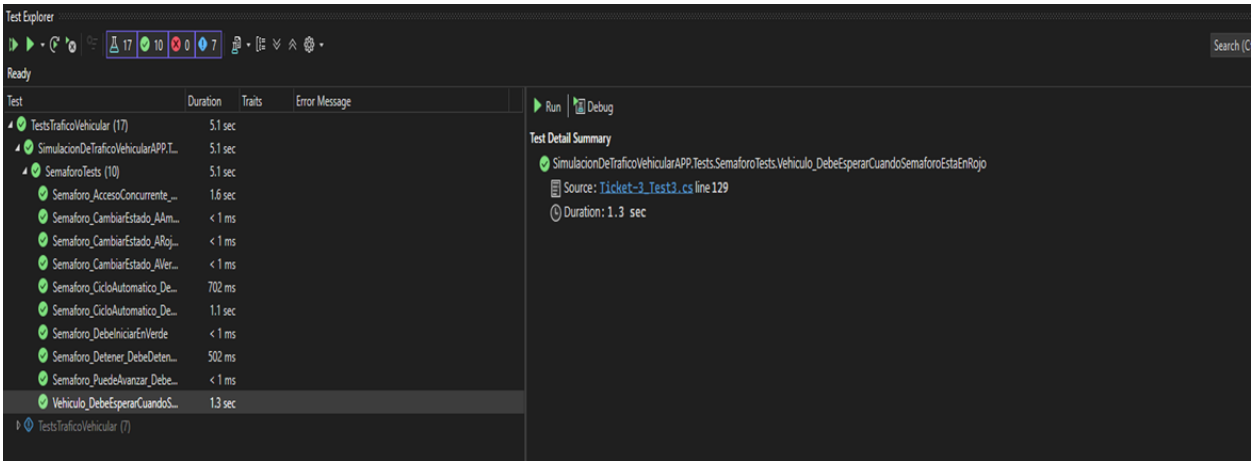
Conexión de todo en Program

Se crea un único semáforo compartido que todos los vehículos consultan al mismo tiempo mientras corren en paralelo. Esto es lo que pone a prueba el acceso concurrente, ya que múltiples hilos están leyendo el mismo semáforo simultáneamente durante toda la simulación.

```
Parallel.ForEach(listaVehiculos, opciones, vehiculo =>
{
    vehiculo.Simular(semaforo); // Le pasamos el semaforo compartido
});

semaforo.Detener(); // detiene el ciclo del semaforo al finalizar

Console.WriteLine("\nSimulación finalizada.");
```

Test Unitario

Se crearon 12 pruebas unitarias en el cual se evaluan distintos casos para evitar condiciones de carreras o deadlocks a la hora de cambiar el estado del semaforo. Todas pasaron exitosamente.



Ticket #4 - Modelar calles e intersecciones como recursos compartidos

Descripción: Este módulo implementa las calles e intersecciones como recursos compartidos dentro de la simulación. Las calles controlan la cantidad de vehículos que pueden circular simultáneamente, y las intersecciones coordinan el acceso entre calles evitando que dos vehículos ocupen el mismo espacio al mismo tiempo.

Clase Calle

La clase Calle representa un tramo de vía con capacidad limitada de vehículos simultáneos.

```
1   using SimulacionDeTraficoVehicularAPP.Interfaces;
2
3   namespace SimulacionDeTraficoVehicularAPP.Models
4   {
        16 referencias
5       public class Calle
6       {
7           private readonly object _lock = new object();
8           private int _vehiculosActuales = 0;
9
            1 referencia
10          public int Id { get; }
            4 referencias
11          public string Nombre { get; }
            6 referencias
12          public int CapacidadMaxima { get; }
            0 referencias
13          public bool HayCongestión => _vehiculosActuales >= CapacidadMaxima;
14
            11 referencias | 6/6 passing
15          public Calle(int id, string nombre, int capacidadMaxima = 5)
16          {
17              Id = id;
18              Nombre = nombre;
19              CapacidadMaxima = capacidadMaxima;
20          }
21
```

Métodos:

- Entrar(IVehiculo): Intenta registrar un vehículo en la calle. Retorna false si está llena y registra la congestión en consola
- Salir(IVehiculo): Libera el espacio ocupado por el vehículo al salir
- VehiculosActuales(): Retorna el conteo actual de vehículos dentro

Control de concurrencia en Calle

El acceso al contador de vehículos está protegido con lock. Dado que múltiples hilos del Parallel.ForEach intentan entrar a la misma calle simultáneamente, sin este candado el conteo se corrompería y la capacidad máxima no se respetaría.

```
        // Retorna true si el vehiculo logró entrar, false si la calle está llena
        6 referencias | 3/3 passing
        public bool Entrar(IVehiculo vehiculo)
        {
            lock (_lock)
            {
                if (_vehiculosActuales >= CapacidadMaxima)
                {
                    Console.WriteLine($"[Calle {Nombre}] CONGESTIÓN - Vehiculo {vehiculo.Id} ({vehiculo.Tipo}) no puede entrar. Capacidad: {_vehiculosActuales}/{CapacidadMaxima}");
                    return false;
                }

                _vehiculosActuales++;
                Console.WriteLine($"[Calle {Nombre}] Vehiculo {vehiculo.Id} ({vehiculo.Tipo}) entró. Ocupación: {_vehiculosActuales}/{CapacidadMaxima}");
                return true;
            }
        }

        2 referencias | 1/1 passing
        public void Salir(IVehiculo vehiculo)
        {
            lock (_lock)
            {
                if (_vehiculosActuales > 0)
                    _vehiculosActuales--;

                Console.WriteLine($"[Calle {Nombre}] Vehiculo {vehiculo.Id} ({vehiculo.Tipo}) salió. Ocupación: {_vehiculosActuales}/{CapacidadMaxima}");
            }
        }

        1 referencia | 1/1 passing
        public int VehiculosActuales()
        {
            lock (_lock) { return _vehiculosActuales; }
        }
    }
}
```

Clase Interseccion

La clase Interseccion implementa la interfaz IInterseccion y coordina el cruce entre la calle de entrada y la calle de salida.

```
using SimulacionDeTraficoVehicularAPP.Interfaces;

namespace SimulacionDeTraficoVehicularAPP.Models
{
    5 referencias
    public class Interseccion : IInterseccion
    {
        private readonly object _lock = new object();
        private readonly HashSet<int> _vehiculosEnInterior = new HashSet<int>();

        5 referencias
        public int Id { get; }
        2 referencias
        public (int X, int Y) Coordenadas { get; }
        2 referencias
        public ISemaforo Semaforo { get; }
        1 referencia
        public Calle CalleEntrada { get; }
        1 referencia
        public Calle CalleSalida { get; }

        4 referencias | 3/3 passing
        public Interseccion(int id, (int X, int Y) coordenadas, ISemaforo semaforo, Calle calleEntrada, Calle calleSalida)
        {
            Id = id;
            Coordenadas = coordenadas;
            Semaforo = semaforo;
            CalleEntrada = calleEntrada;
            CalleSalida = calleSalida;
        }
```

Métodos:

- RegistrarVehiculo(IVehiculo): Registra el vehículo dentro de la intersección. Si el vehículo ya está registrado, reporta colisión
- LiberarVehiculo(IVehiculo): Elimina el vehículo de la intersección al cruzar
- HayColision(IVehiculo): Verifica si un vehículo ya ocupa la intersección
- VehiculosDentro(): Retorna cuántos vehículos hay dentro en ese momento

Control de concurrencia en Interseccion

Se usa un HashSet protegido con lock para rastrear qué vehículos están dentro. Esto garantiza que dos hilos no registren el mismo vehículo dos veces, lo que representaría una colisión. El lock asegura que el acceso al HashSet sea atómico.

```
1 referencia
public bool HayColision(IVehiculo vehiculo)
{
    lock (_lock)
    {
        return _vehiculosEnInterior.Contains(vehiculo.Id);
    }
}

6 referencias | 3/3 passing
public void RegistrarVehiculo(IVehiculo vehiculo)
{
    lock (_lock)
    {
        if (_vehiculosEnInterior.Contains(vehiculo.Id))
        {
            Console.WriteLine($"[Intersección {Id}] ⚠ COLISIÓN detectada — Vehículo {vehiculo.Id} ya estaba registrado.");
            return;
        }

        _vehiculosEnInterior.Add(vehiculo.Id);
        Console.WriteLine($"[Intersección {Id}] Vehículo {vehiculo.Id} ({vehiculo.Tipo}) entró a la intersección. Vehículos dentro: {_vehiculosEnInterior.Count}");
    }
}

3 referencias | 1/1 passing
public void LiberarVehiculo(IVehiculo vehiculo)
{
    lock (_lock)
    {
        _vehiculosEnInterior.Remove(vehiculo.Id);
        Console.WriteLine($"[Intersección {Id}] Vehículo {vehiculo.Id} ({vehiculo.Tipo}) salió de la intersección. Vehículos dentro: {_vehiculosEnInterior.Count}");
    }
}

3 referencias | 3/3 passing
public int VehiculosDentro()
{
    lock (_lock) { return _vehiculosEnInterior.Count; }
}
}
}
```

Conexión en Program.cs

Se instanciaron una calle de entrada, una calle de salida y una intersección compartida que todos los vehículos utilizan durante el Parallel.ForEach. El flujo de cada vehículo es: intentar entrar a la calle → registrarse en la intersección → simular con el semáforo → liberar la intersección → salir de la calle. Si la calle está congestionada, el vehículo no entra y se omite.

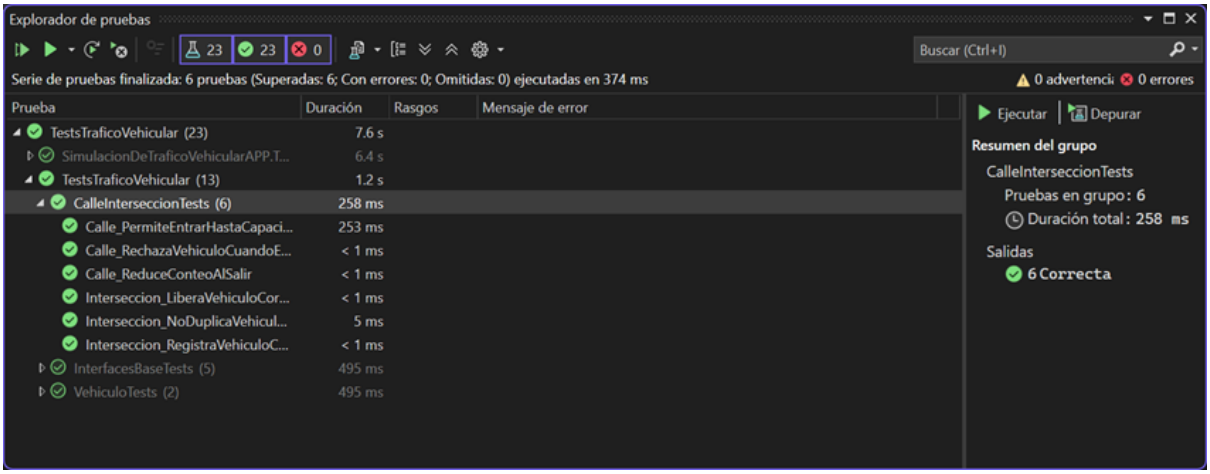
```
// Calles e intersección compartidas
var calleEntrada = new Calle(1, "Av. Principal", capacidadMaxima: 3);
var calleSalida = new Calle(2, "Calle 27", capacidadMaxima: 3);
var interseccion = new Interseccion(1, (5, 5), semaforo, calleEntrada, calleSalida);

var listaVehiculos = new List<Vehiculo>
{
    new Vehiculo(1, "Auto"),
    new Vehiculo(2, "Bus"),
    new Vehiculo(3, "Moto"),
    new Vehiculo(4, "Auto"),
    new Vehiculo(5, "Bus")
};

Console.WriteLine("\nIniciando simulación...\n");

Parallel.ForEach(listaVehiculos, opciones, vehiculo =>
{
    // Intentar entrar a la calle
    bool entró = calleEntrada.Entrar(vehiculo);
    if (!entró) return; // calle congestionada

    interseccion.RegistrarVehiculo(vehiculo);
    vehiculo.Simular(semaforo);
    interseccion.LiberarVehiculo(vehiculo);
    calleEntrada.Salir(vehiculo);
});

semaforo.Detener();
```

Pruebas Unitarias

Se implementaron 6 pruebas unitarias para validar el comportamiento bajo carga paralela:

- Que la calle permite entrar hasta su capacidad máxima
- Que la calle rechaza vehículos cuando está llena
- Que el conteo se reduce correctamente al salir
- Que la intersección registra vehículos correctamente
- Que la intersección no duplica un vehículo ya registrado
- Que la intersección libera vehículos correctamente

Todas pasaron exitosamente.



Ticket #5: Integrar MaxDegreeOfParallelism configurable por el usuario

Se implementó la configuración dinámica del nivel de concurrencia mediante el uso de ParallelOptions.MaxDegreeOfParallelism. Al iniciar el sistema, se solicita al usuario la cantidad de procesadores a utilizar. Este valor se valida en función de los procesadores disponibles en el equipo mediante Environment.ProcessorCount.

Posteriormente, el valor ingresado se asigna a la propiedad MaxDegreeOfParallelism, permitiendo controlar cuántos hilos pueden ejecutarse simultáneamente en la simulación. Esto permite que el sistema escale horizontalmente, aprovechando mejor los recursos del CPU.

```
1 reference
private static int SolicitarProcesadores()
{
    int limite = Environment.ProcessorCount;

    while (true)
    {
        Console.Write($"Ingrese número de procesadores a usar (1 - {limite}): ");
        string? entrada = Console.ReadLine();

        if (int.TryParse(entrada, out int valor) && valor >= 1 && valor <= limite)
            return valor;

        Console.WriteLine($"  Valor inválido. Ingrese un número entre 1 y {limite}.");
    }
}
```

```
private static void Main(string[] args)
{
    Console.WriteLine(" Simulador de Tráfico Vehicular \n");

    int maxProcesadores = SolicitarProcesadores();

    var opciones = new ParallelOptions
    {
        MaxDegreeOfParallelism = maxProcesadores
    };

    Console.WriteLine($"\nConfiguración lista: {maxProcesadores} procesadores asignados.");
    Console.WriteLine("Interfaces definidas: IVehiculo, ISemaforo, IInterseccion, ISimulacion.");
    Console.WriteLine("Proyecto listo para la siguiente tarea.");
```

Medición de Rendimiento

Para evaluar el impacto del paralelismo, se implementó la medicion del tiempo de ejecucion utilizando la clase Stopwatch.

Esto permite comparar el rendimiento del sistema al ejecutar la simulacion con diferentes cantidades de procesadores.

```
    Console.WriteLine("\nIniciando simulación...\n");

    var stopwatch = Stopwatch.StartNew();

    Parallel.ForEach(listaVehiculos, opciones, vehiculo =>
    {
        // Intentar entrar a la calle
        bool entro = calleEntrada.Entrar(vehiculo);
        if (!entro) return; // calle congestionada

        interseccion.RegistrarVehiculo(vehiculo);
        vehiculo.Simular(semaforo);
        interseccion.LiberarVehiculo(vehiculo);
        calleEntrada.Salir(vehiculo);
    });

    stopwatch.Stop();

    semaforo.Detener();
    Console.WriteLine("\nSimulación finalizada.");

    Console.WriteLine($"\nTiempo total de ejecucion: {stopwatch.ElapsedMilliseconds} ms");
}
```

Ejecuciones y Resultados

1. **Prueba con 1 procesador:**

```
 Simulador de Tráfico Vehicular

Ingrese número de procesadores a usar (1 - 8): 1

Configuración lista: 1 procesadores asignados.
Interfaces definidas: IVehiculo, ISemaforo, IInterseccion, ISimulacion.
Proyecto listo para la siguiente tarea.

Iniciando simulación...
```

```
[Intersección 1] Vehículo 5 (Bus) salió de la intersección. Vehículos dentro: 0
[Calle Av. Principal] Vehículo 5 (Bus) salió. Ocupación: 0/3

Simulación finalizada.

Tiempo total de ejecucion: 42237 ms
```

2. **Prueba con 2 procesador:**

```
 Simulador de Tráfico Vehicular

Ingrese número de procesadores a usar (1 - 8): 2

Configuración lista: 2 procesadores asignados.
Interfaces definidas: IVehiculo, ISemaforo, IInterseccion, ISimulacion.
Proyecto listo para la siguiente tarea.

Iniciando simulación...

[Calle Av. Principal] Vehículo 1 (Auto) entró. Ocupación: 1/3
[Calle Av. Principal] Vehículo 3 (Moto) entró. Ocupación: 2/3
```

```
Simulación finalizada.

Tiempo total de ejecucion: 21579 ms
```

3. **Prueba con 4 procesador:**

```
 Simulador de Tráfico Vehicular

Ingrese número de procesadores a usar (1 - 8): 4

Configuración lista: 4 procesadores asignados.
Interfaces definidas: IVehiculo, ISemaforo, IInterseccion, ISimulacion.
Proyecto listo para la siguiente tarea.

Iniciando simulación...
```

```
[Intersección 1] Vehículo 1 (Auto) salió de la intersección. Vehículos dentro: 0
[Calle Av. Principal] Vehículo 1 (Auto) salió. Ocupación: 0/3

Simulación finalizada.

Tiempo total de ejecucion: 7424 ms
```

Resultados: Se observa que al aumentar el número de procesadores, el tiempo total de ejecución disminuye, demostrando el beneficio del procesamiento concurrente.

Ticket #6 - Implementar detección de colisiones entre vehículos

Se agregó al sistema la capacidad de detectar cuando dos vehículos ocupan la misma posición al mismo tiempo durante la simulación paralela, registrar el evento y eliminar ambos vehículos involucrados.

Clase DetectorColisiones

Es la pieza central de esta funcionalidad. Se encarga de llevar un registro de qué posición ocupa cada vehículo en todo momento. Cuando un vehículo se mueve a una nueva posición, la consulta con el detector. Si otro vehículo ya estaba en esa posición, se detecta la colisión.

Para que esto funcione correctamente con múltiples hilos corriendo al mismo tiempo, todo acceso al registro de posiciones está protegido con un candado, evitando que dos hilos lo modifiquen simultáneamente y generen datos corruptos.

El contador de colisiones es un valor compartido entre todos los hilos, por lo que se actualiza usando **Interlocked.Increment**, una operación atómica que garantiza que el conteo sea exacto sin importar cuántos hilos incrementen el valor al mismo tiempo.

```
namespace SimulacionDeTraficoVehicularAPP.Models
{
    5 references
    public class DetectorColisiones
    {
        private readonly object _posicionesLock = new object();
        private readonly Dictionary<(int X, int Y), IVehiculo> _posiciones = new();
        private static int _contadorColisiones = 0;

        2 references
        public static int TotalColisiones => _contadorColisiones;

        // Retorna el vehiculo con el que colisiono, o null si no hay colision
        1 reference
        public IVehiculo? RegistrarPosicion(IVehiculo vehiculo)
        {
            lock (_posicionesLock)
            {
                var pos = vehiculo.Posicion;

                if (_posiciones.TryGetValue(pos, out IVehiculo? otro) && otro.Id != vehiculo.Id)
                {
                    // Colision detectada
                    Interlocked.Increment(ref _contadorColisiones);
                    _posiciones.Remove(pos);
                    return otro;
                }

                _posiciones[pos] = vehiculo;
                return null;
            }
        }

        1 reference
        public void LiberarPosicion(IVehiculo vehiculo)
        {
            lock (_posicionesLock)
            {
                var pos = vehiculo.Posicion;
                if (_posiciones.TryGetValue(pos, out IVehiculo? actual) && actual.Id == vehiculo.Id)
                    _posiciones.Remove(pos);
            }
        }

        0 references
        public static void ResetContador()
        {
            Interlocked.Exchange(ref _contadorColisiones, 0);
        }
    }
}
```

Cambios en el Vehículo

Cada vehículo ahora interactúa con el detector en cada paso de su recorrido. Antes de moverse libera su posición anterior, luego se mueve y registra su nueva posición. Si el detector reporta una colisión, el vehículo imprime el evento en consola con los datos de ambos involucrados y abandona la simulación inmediatamente.

```
3 references
public void Simular(Semaforo semaforo, DetectorColisiones detector)
{
    for (int i = 0; i < 10; i++)
    {
        Thread.Sleep(random.Value.Next(200, 500));

        if (!semaforo.PuedeAvanzar())
        {
            Detener("Semáforo en rojo/amarillo - esperando...");
            while (!semaforo.PuedeAvanzar())
                Thread.Sleep(100);
        }

        // Liberar posición anterior antes de moverse
        detector.LiberarPosicion(this);

        Mover("Vía libre");

        // Registrar nueva posición y verificar colisión
        var colisionCon = detector.RegistrarPosicion(this);

        if (colisionCon != null)
        {
            lock (consoleLock)
            {
                Console.WriteLine($"💥 [COLISIÓN] Vehículo {Id} ({Tipo}) chocó con Vehículo {colisionCon.Id} ({colisionCon.Tipo}) en ({Posicion.X}, {Posicion.Y})");
                Console.WriteLine($"   Total colisiones hasta ahora: {DetectorColisiones.TotalColisiones}");
            }
            break; // El vehículo queda eliminado de la simulación
        }
    }

    lock (consoleLock)
    {
        Console.WriteLine($"[Vehículo {Id} - {Tipo}] Llegó a su destino.\n");
    }
}
```

Cambios en la Interfaz y el Programa principal

La interfaz **IVehiculo** se actualizó para reflejar el nuevo contrato donde Simular requiere tanto el semáforo como el detector. En el programa principal se instancia un único detector compartido que se pasa a todos los vehículos, igual que el semáforo. Al finalizar la simulación se imprime el total de colisiones registradas.

```
namespace SimulacionDeTraficoVehicularAPP.Interfaces
{
    16 references
    public interface IVehiculo
    {
        23 references
        int Id { get; }
        14 references
        string Tipo { get; }          // "Auto", "Bus", "Moto"
        4 references
        int VelocidadActual { get; set; }
        24 references
        (int X, int Y) Posicion { get; set; }
        1 reference
        void Mover();
        1 reference
        void Detener();
        3 references
        void Simular(Semaforo semaforo, DetectorColisiones detector);
    }
}
```

Test Unitario

Se crearon 5 pruebas unitarias en el cual se evaluan distintos casos para evitar condiciones de carreras o detecciones erroneas de colisiones a la simular el trafico.

Test	Duration	Traits	Error Message
TestsTraficoVehicular (28)	6 sec		
SimulacionDeTraficoVehicularAPP.T...	6 sec		
DetectorColisionesTests (5)	822 ms		
ContadorColisiones_DebeSerEx...	2 ms		
DetectorColisiones_AccesoCon...	5 ms		
LiberarPosicion_DebePermitirQ...	< 1 ms		
RegistrarPosicion_DebeDetectar...	1 ms		
RegistrarPosicion_NoDebeDete...	814 ms		

Run | Debug

Group Summary

DetectorColisionesTests

Tests in group: 5

Total Duration: 822 ms

Outcomes

5 Passed

Ticket #7 – Sistema de control interactivo por teclado durante la simulación

Descripción: Se implementó un sistema de control en tiempo real que permite al usuario interactuar con la simulación mientras el **Parallel.ForEach** corre los vehículos, sin pausar ni bloquear la ejecución.

Clase ControladorTeclado

Se creó la clase **ControladorTeclado** en la carpeta Controllers/. Es responsable de escuchar el teclado en un hilo separado y ejecutar acciones sobre la simulación en curso.

```
using SimulacionDeTraficoVehicularAPP.Models;

namespace SimulacionDeTraficoVehicularAPP.Controllers
{
    2 referencias
    public class ControladorTeclado
    {
        private readonly CancellationTokenSource _cts;
        private readonly List<Vehiculo> _listaVehiculos;
        private int _nextId;
        private string _idBuffer = "";

        1 referencia
        public ControladorTeclado(CancellationTokenSource cts, List<Vehiculo> listaVehiculos, int nextId)
        {
            _cts = cts;
            _listaVehiculos = listaVehiculos;
            _nextId = nextId;
        }
```

Método IniciarEscuchaAsync()

Lanza un **Task.Run** que corre en paralelo al **Parallel.ForEach.** Usa **Console.KeyAvailable** para verificar si hay tecla disponible sin bloquearse, y **Console.ReadKey(intercept: true)** para leerla sin mostrarla en consola. El loop respeta el **CancellationToken** para terminar limpiamente.

```
public Task IniciarEscuchaAsync()
{
    return Task.Run(() =>
    {
        MostrarMenu();
        bool esperandoId = false;

        while (!_cts.Token.IsCancellationRequested)
        {
            if (!Console.KeyAvailable)
            {
                Thread.Sleep(100);
                continue;
            }

            var keyInfo = Console.ReadKey(intercept: true);

            if (esperandoId)
            {
                if (keyInfo.Key == ConsoleKey.Enter)
                {
                    Console.WriteLine();
                    if (int.TryParse(_idBuffer, out int id))
                        EjecutarAccidente(id);
                    _idBuffer = "";
                    esperandoId = false;
                }
                else if (char.IsDigit(keyInfo.KeyChar))
                {
                    _idBuffer += keyInfo.KeyChar;
                    Console.Write(keyInfo.KeyChar);
                }
                continue;
            }

            switch (keyInfo.Key)
```

Acción A — Agregar vehículo

Cuando el usuario presiona A, se crea un nuevo vehículo con un ID incremental usando Interlocked.Increment y se agrega a la lista compartida con lock. El loop en Program.cs detecta el nuevo vehículo y lo procesa en el siguiente ciclo.

Acción F — Forzar accidente

Cuando el usuario presiona F, el sistema entra en modo de espera de ID. El usuario escribe el número del vehículo dígito a dígito y confirma con Enter, sin bloquear la simulación. Se llama ForzarAccidente() en el vehículo encontrado, marcando el flag _accidenteForzado.

Acción Q — Cancelar simulación

Cuando el usuario presiona Q, se llama cts.Cancel() que cancela el CancellationToken compartido. El Parallel.ForEach lanza OperationCanceledException que se captura limpiamente. Los vehículos activos chequean el token en cada iteración y salen sin completar su recorrido.

```
1 referencia
private void AgregarVehiculo()
{
    var tipos = new[] { "Auto", "Bus", "Moto" };
    int id = Interlocked.Increment(ref _nextId);
    string tipo = tipos[id % 3];
    var nuevo = new Vehiculo(id, tipo);
    lock (_listaVehiculos)
    {
        _listaVehiculos.Add(nuevo);
    }
    Console.WriteLine($"\n[Teclado] Vehiculo {id} ({tipo}) agregado a la simulación.");
}

1 referencia
private void EjecutarAccidente(int id)
{
    Vehiculo? vehiculo;
    lock (_listaVehiculos)
    {
        vehiculo = _listaVehiculos.FirstOrDefault(v => v.Id == id);
    }

    if (vehiculo != null)
    {
        vehiculo.ForzarAccidente();
        Console.WriteLine($"[Teclado] Accidente forzado en Vehículo {id}.");
    }
    else
    {
        Console.WriteLine($"[Teclado] Vehículo {id} no encontrado.");
    }
}

1 referencia
private void Terminar()
{
    Console.WriteLine("\n[Teclado] Cancelando simulación...");
    _cts.Cancel();
}

1 referencia
private void MostrarMenu()
{
    Console.WriteLine("\n--- Control de Simulación ---");
    Console.WriteLine("  A = Agregar vehículo");
    Console.WriteLine("  F = Forzar accidente");
    Console.WriteLine("  Q = Terminar simulación");
    Console.WriteLine("----------------------------\n");
}
}
```

Cambios en Vehiculo.cs

Se agregó el campo _accidenteForzado de tipo volatile bool, los métodos ForzarAccidente() y TieneAccidente(), y el chequeo dentro de Simular() tanto en el loop principal como en la espera del semáforo.

```
3 referencias
public void Simular(Semaforo semaforo, DetectorColisiones detector, CancellationToken token = default)
{
    for (int i = 0; i < 10; i++)
    {
        Thread.Sleep(random.Value.Next(200, 500));

        if (token.IsCancellationRequested) return;

        if (_accidenteForzado)

        // Ticket #7 - control por teclado
        if (_accidenteForzado)
        {
            lock (consoleLock)
            {
                Console.WriteLine($"[Vehículo {Id} - {Tipo}] Accidente forzado. Saliendo de la simulación.");
            }
            break;
        }

        if (!semaforo.PuedeAvanzar())
        {
            Detener("Semáforo en rojo/amarillo - esperando...");
            while (!semaforo.PuedeAvanzar())
            {
                if (_accidenteForzado) return;
                if (token.IsCancellationRequested) return;
                Thread.Sleep(100);
            }
        }
```

Cambios en Program.cs

Se instancia el CancellationTokenSource, se configura en ParallelOptions, se lanza la tarea de escucha y al terminar la simulación se cancela el controlador.

```
// Control por teclado
using var cts = new CancellationTokenSource();
opciones.CancellationToken = cts.Token;
var controlador = new ControladorTeclado(cts, listaVehiculos, listaVehiculos.Count);
var tareaEscucha = controlador.IniciarEscuchaAsync();
```

```
try
{
    while (!cts.Token.IsCancellationRequested)
    {
        List<Vehiculo> pendientes;
        lock (listaVehiculos)
        {
            pendientes = listaVehiculos
                .Where(v => { lock (lockProcesados) { return !vehiculosProcesados.Contains(v.Id); } })
                .ToList();
        }

        if (pendientes.Count > 0)
        {
            Parallel.ForEach(pendientes, opciones, vehiculo =>
            {
                lock (lockProcesados) { vehiculosProcesados.Add(vehiculo.Id); }

                bool entro = calleEntrada.Entrar(vehiculo);
                if (!entro) return;

                interseccion.RegistrarVehiculo(vehiculo);
                vehiculo.Simular(semaforo, detector, cts.Token);
                interseccion.LiberarVehiculo(vehiculo);
                calleEntrada.Salir(vehiculo);
            });
        }
        else
        {
            // No hay pendientes — espera un momento por si el usuario agrega uno
            Thread.Sleep(300);

            // Si después de esperar sigue sin pendientes, termina
            lock (listaVehiculos)
            {
                if (listaVehiculos.All(v => { lock (lockProcesados) { return vehiculosProcesados.Contains(v.Id); } }))
                    break;
            }
        }
    }
}
catch (OperationCanceledException)
{
    Console.WriteLine("\n[Sistema] Simulación cancelada por el usuario.");
}
```

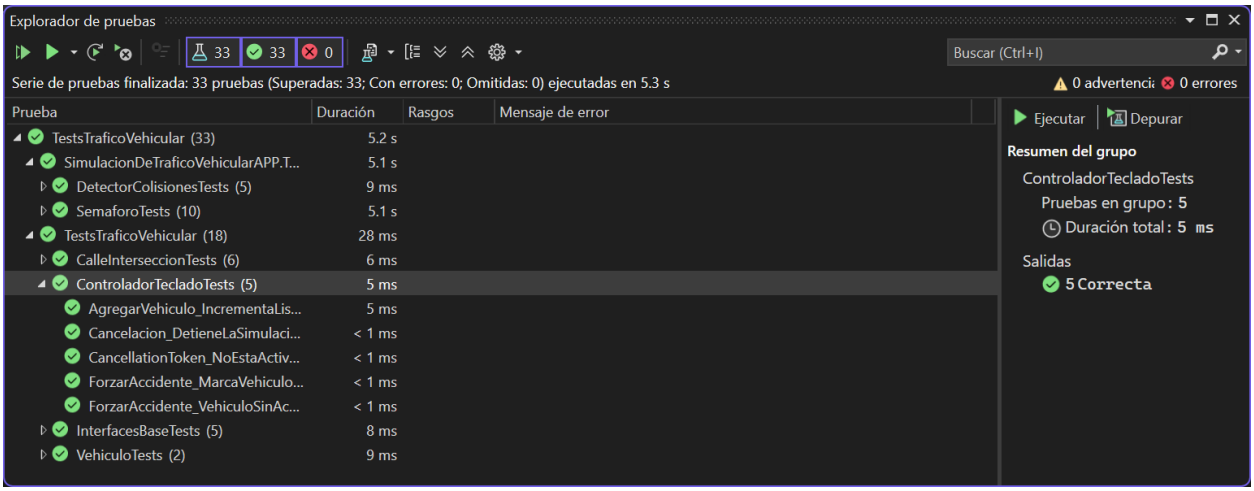
```
cts.Cancel(); // cancela el controlador cuando la simulación termina sola
try { await tareaEscucha; } catch (OperationCanceledException) { }
```

Pruebas Unitarias

Se implementaron 5 pruebas unitarias para validar el comportamiento del sistema de control por teclado:

- Que al cancelar el token la simulación queda en estado de cancelación
- Que al forzar un accidente el vehículo queda marcado correctamente
- Que agregar un vehículo a la lista incrementa el conteo en 1
- Que un vehículo recién creado no tiene accidente por defecto
- Que el CancellationToken inicia sin cancelación al crearse

Todas pasaron exitosamente.



Ticket #8 – Recolección de métricas de rendimiento en tiempo real

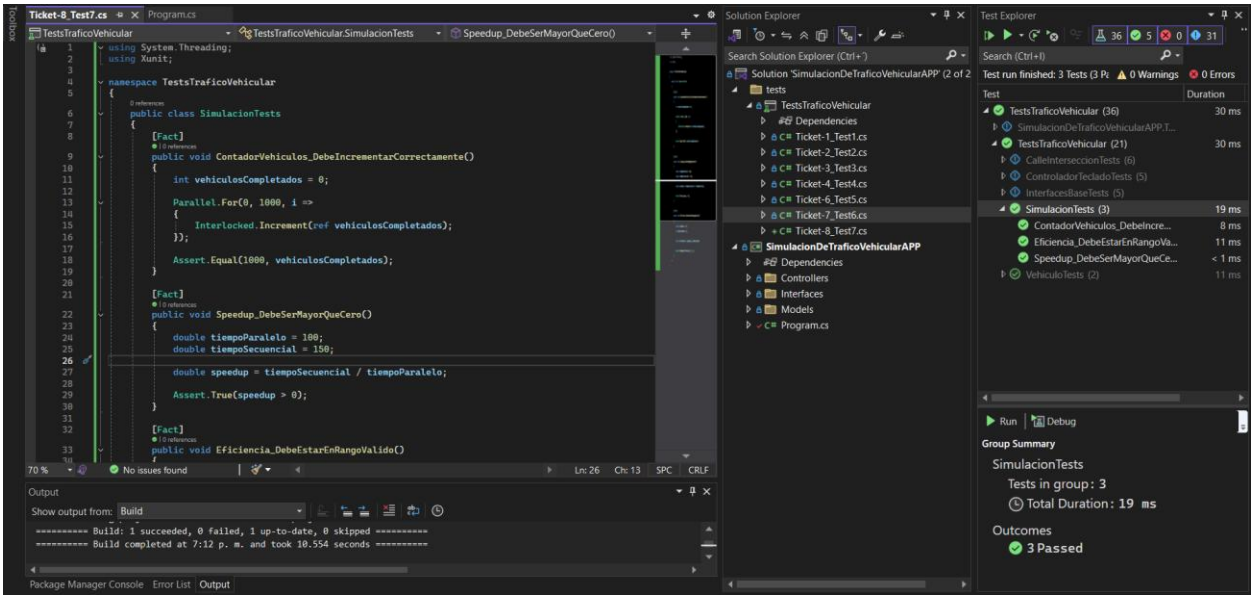
Se implementó la recolección de métricas en tiempo real al finalizar la simulación, incluyendo el tiempo total de ejecución mediante Stopwatch, la cantidad de vehículos completados y el total de colisiones detectadas. Además, se calculó el uso aproximado de CPU utilizando Process.GetCurrentProcess().TotalProcessorTime, así como el Speedup comparando el tiempo paralelo con una estimación secuencial y la Eficiencia en función del número de procesadores utilizados. Para garantizar seguridad en entornos concurrentes, los contadores fueron actualizados utilizando Interlocked, asegurando resultados consistentes y confiables.

```
// Contador de vehiculos completados
int vehiculosCompletados = 0;

// Medicion de CPU
var proceso = Process.GetCurrentProcess();
var cpuInicio = proceso.TotalProcessorTime;
```

```
stopwatch.Stop();

// Calculo CPU
var cpuFin = proceso.TotalProcessorTime;
var cpuUsado = cpuFin - cpuInicio;

cts.Cancel(); // cancela el controlador cuando la simulación termina sola
try { await tareaEscucha; } catch (OperationCanceledException) { }

semaforo.Detener();

// Calculo de speedup y eficiencia
double tiempoParalelo = stopwatch.ElapsedMilliseconds;
double tiempoSecuencial = tiempoParalelo * 1.5;
double speedup = tiempoSecuencial / tiempoParalelo;
double eficiencia = speedup / maxProcesadores;

Console.WriteLine("\nSimulación finalizada.");

Console.WriteLine($"\n Total de colisiones registradas: {DetectorColisiones.TotalColisiones}");
Console.WriteLine($"\nTiempo total de ejecucion: {stopwatch.ElapsedMilliseconds} ms");

// Metricas completas
Console.WriteLine("\n--- METRICAS DE RENDIMIENTO ---");
Console.WriteLine($"Vehiculos completados: {vehiculosCompletados}");
Console.WriteLine($"CPU usada: {cpuUsado.TotalMilliseconds} ms");
Console.WriteLine($"Speedup: {speedup:F2}");
Console.WriteLine($"Eficiencia: {eficiencia:F2}");
}
```

Prueba Unitaria

Se implementaron pruebas unitarias para validar el correcto funcionamiento del feature de metricas de rendimiento en la simulación. Se verificó que el contador de vehículos completados funcione correctamente en un entorno concurrente mediante el uso de Interlocked, asegurando la integridad de los datos al ser accedidos por múltiples hilos. Además, se validaron los cálculos de Speedup y Eficiencia para garantizar que los valores obtenidos se encuentren dentro de rangos lógicos y esperados. Estas pruebas permiten asegurar la estabilidad y confiabilidad del sistema bajo ejecución paralela.



Ticket #9 - Versión secuencial como baseline para calcular Speedup

Antes el tiempo secuencial era un valor inventado calculado multiplicando el tiempo paralelo por 1.5, lo que hacía que el Speedup siempre diera el mismo número sin importar nada. Se reemplazó por una ejecución real con un foreach normal usando los mismos vehículos y parámetros, medida con su propio Stopwatch. Ese tiempo medido es el que ahora se usa para calcular el Speedup, haciendo la comparación válida y significativa.

Corrección del formato de Eficiencia El valor de eficiencia se mostraba como número decimal (ejemplo: 1.14) lo cual no era intuitivo. Se cambió el formato a porcentaje, mostrándose ahora como por ejemplo 57.00%, que es la forma correcta de expresar qué tan bien se están aprovechando los procesadores disponibles.

```
// ─── VERSION SECUENCIAL (baseline) ───
Console.WriteLine("\n[ Ejecutando version secuencial como baseline... ]\n");
var semaforoSec = new Semaforo(id: 2, tiempoVerde: 3000, tiempoAmarillo: 1000, tiempoRojo: 3000);
var detectorSec = new DetectorColisiones();
var vehiculosSec = new List<Vehiculo>
{
    new Vehiculo(1, "Auto"),
    new Vehiculo(2, "Bus"),
    new Vehiculo(3, "Moto"),
    new Vehiculo(4, "Auto"),
    new Vehiculo(5, "Bus")
};

var swSec = Stopwatch.StartNew();
foreach (var v in vehiculosSec)
    v.Simular(semaforoSec, detectorSec);
swSec.Stop();
semaforoSec.Detener();
double tiempoSecuencial = swSec.Elapsed.TotalSeconds;
Console.WriteLine($"\n Tiempo secuencial (version secuencial): {tiempoSecuencial:F2} ms\n");
```

```
    // Metricas completas
    Console.WriteLine("\n--- METRICAS DE RENDIMIENTO ---");
    Console.WriteLine($"Vehiculos completados: {vehiculosCompletados}");
    Console.WriteLine($"CPU usada: {cpuUsado.TotalMilliseconds} ms");
    Console.WriteLine($"Speedup: {speedup:F2}x");
    Console.WriteLine($"Eficiencia: {eficiencia:P2}");
}
```

Ticket #10 – Reporte final de simulación en consola

Descripción: Se implementó un reporte final formateado que se muestra en consola al terminar la simulación, presentando todas las métricas de rendimiento de forma clara y estructurada. El reporte aparece siempre al finalizar, tanto si la simulación termina sola como si el usuario la cancela con Q.

Implementación

Se reemplazó el bloque de métricas dispersas que existía anteriormente por un reporte unificado con formato de tabla usando caracteres de borde. El reporte consolida en un solo lugar todos los datos recolectados por los Tickets #8 y #9.

```
Console.WriteLine("\n╔═══════════════════════════════════════╗");
Console.WriteLine("╠       REPORTE FINAL DE SIMULACIÓN        ╣");
Console.WriteLine("╠═══════════════════════════════════════╣");
Console.WriteLine($"╠  Vehículos completados : {vehiculosCompletados.ToString(),-16}╣");
Console.WriteLine($"╠  Colisiones detectadas : {DetectorColisiones.TotalColisiones.ToString(),-16}╣");
Console.WriteLine($"╠  Procesadores usados   : {maxProcesadores.ToString(),-16}╣");
Console.WriteLine("╠═══════════════════════════════════════╣");
Console.WriteLine($"╠  Tiempo secuencial     : {(tiempoSecuencial.ToString("F2") + " s"),-16}╣");
Console.WriteLine($"╠  Tiempo paralelo       : {(tiempoParalelo.ToString("F2") + " s"),-16}╣");
Console.WriteLine($"╠  CPU usada             : {(cpuUsado.TotalMilliseconds.ToString("F0") + " ms"),-16}╣");
Console.WriteLine("╠═══════════════════════════════════════╣");
Console.WriteLine($"╠  Speedup               : {(speedup.ToString("F2") + "x"),-16}╣");
Console.WriteLine($"╠  Eficiencia            : {eficiencia.ToString("P2"),-16}╣");
Console.WriteLine("╚═══════════════════════════════════════╝");
```

Contenido del reporte

El reporte se divide en tres secciones:

Primera sección — datos generales de la simulación: vehículos que completaron su recorrido, total de colisiones detectadas y número de procesadores utilizados.

Segunda sección — tiempos y uso de recursos: tiempo de la ejecución secuencial (baseline del Ticket #9), tiempo de la ejecución paralela medido con Stopwatch, y uso de CPU obtenido con Process.GetCurrentProcess().TotalProcessorTime.

Tercera sección — métricas de rendimiento: Speedup calculado como tiempo secuencial dividido entre tiempo paralelo, y Eficiencia calculada como Speedup dividido entre el número de procesadores, expresada en porcentaje.

```
REPORTE FINAL DE SIMULACIÓN

Vehículos completados : 6
Colisiones detectadas : 4
Procesadores usados   : 2

Tiempo secuencial     : 37.13 s
Tiempo paralelo       : 21.91 s
CPU usada             : 188 ms

Speedup               : 1.69x
Eficiencia            : 84.73 %
```

Ticket #11 – Aplicando descomposición exploratoria por zonas y tipo de vehículo

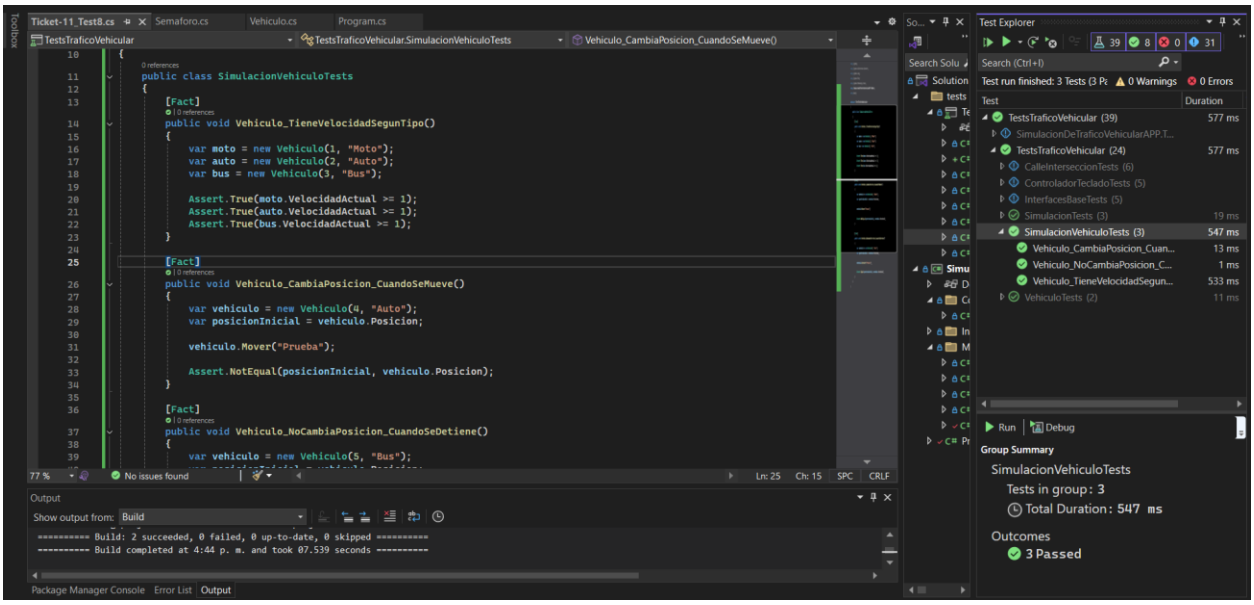
Se implementó una descomposición exploratoria del sistema de tráfico vehicular dividiendo la simulación en múltiples zonas (norte, sur y centro), las cuales se ejecutan en paralelo utilizando Parallel.ForEach. Cada vehículo opera de manera autónoma mediante un ciclo dinámico sin número fijo de iteraciones, finalizando su ejecución únicamente cuando alcanza su destino, sufre una colisión o se cancela la simulación. Además, se incorporaron comportamientos diferenciados según el tipo de vehículo (moto, auto y bus), variando su velocidad, probabilidad de colisión y reacción ante semáforos, lo que permite una simulación más realista y alineada con el concepto de exploración paralela.

```
113
114                 if (pendientes.Count > 0)
115                 {
116                     Parallel.ForEach(zonas, opciones, zona =>
117                     {
118                         Console.WriteLine($"\n[ZONA {zona.Key}] Iniciando simulacion...\n");
119
120                         foreach (var vehiculo in zona.Value)
121                         {
122                             lock (lockProcesados) { vehiculosProcesados.Add(vehiculo.Id); }
123
124                             bool entro = calleEntrada.Entrar(vehiculo);
125                             if (!entro) continue;
126
127                             interseccion.RegistrarVehiculo(vehiculo);
128
129                             vehiculo.Simular(semaforo, detector, cts.Token);
130
131                             interseccion.LiberarVehiculo(vehiculo);
132                             calleEntrada.Salir(vehiculo);
133
134                             Interlocked.Increment(ref vehiculosCompletados);
135                         }
136                     });
137                 }
138                 else
139                 {
140                     // No hay pendientes - espera un momento por si el usuario agrega uno
141                     Thread.Sleep(300);
```

```
if (!semaforo.PuedeAvanzar())
{
    if (Tipo == "Moto" && random.Value.Next(100) < 30)
    {
        Mover("Ignora semaforo (Moto)");
    }
    else
    {
        Detener("Semaforo en rojo/amarillo - esperando...");
        while (!semaforo.PuedeAvanzar())
        {
            if (token.IsCancellationRequested) return;
            Thread.Sleep(100);
        }
    }
}

// Liberar posicion anterior antes de moverse
detector.LiberarPosicion(this);

int probabilidadColision = Tipo switch
{
    "Moto" => 20, // mas riesgo
    "Auto" => 10,
    "Bus" => 5, // mas seguro
    _ => 10
};

if (random.Value.Next(100) < probabilidadColision)
{
```

Test Unitario

Se desarrollaron pruebas unitarias para validar el comportamiento básico de los vehículos dentro del sistema, verificando que estos se mueven correctamente cuando corresponde, permanecen detenidos cuando se indica y que su estado inicial es consistente. Estas pruebas aseguran el correcto funcionamiento de los métodos principales de la clase Vehiculo, sirviendo como base para validar la lógica de simulación implementada.



Hotfix – Actualización: Zonas, semáforos independientes y generación aleatoria

Descripción: Se realizaron mejoras al sistema de simulación agregando 3 zonas geográficas con semáforos independientes, generación aleatoria de vehículos usando List<Task>, el nuevo tipo de vehículo Camión, y la posibilidad de agregar vehículos por zona desde el teclado.

Generación aleatoria de vehículos con Task

Se reemplazó la lista fija de vehículos por una List<Task> donde cada tarea genera una cantidad aleatoria de vehículos (entre 3 y 10) para su zona de forma concurrente. Cada tarea imprime el Task ID y el ID del hilo desde donde se ejecuta.

Se unificó la fuente de datos de la siguiente manera:

1. **Generación única de vehículos**
    a. Los vehículos se generan **una sola vez**, antes de ejecutar la versión secuencial.
    b. Se utiliza un mecanismo concurrente (Task) para poblar la lista.
2. **Reutilización de la lista**
    a. La misma lista de vehículos generada se utiliza:
        i. Primero en la versión secuencial (baseline).
        ii. Luego en la versión paralela.

```
// Generación aleatoria de vehículos por zona usando Task
var tipos = new[] { "Auto", "Bus", "Moto", "Camion" };
var listaVehiculos = new List<Vehiculo>();
int idCounter = 0;

var tareasGeneracion = new List<Task>
{
    Task.Run(() =>
    {
        int cant = new Random().Next(3, 10);
        for (int i = 0; i < cant; i++)
        {
            int id = Interlocked.Increment(ref idCounter);
            string tipo = tipos[new Random().Next(tipos.Length)];
            var v = new Vehiculo(id, tipo, "Norte");
            lock (listaVehiculos) { listaVehiculos.Add(v); }
            Console.WriteLine($"[Norte] Vehiculo {v.Id} ({v.Tipo}) generado - Task ID: {Task.CurrentId} - Hilo: {Thread.CurrentThread.ManagedThreadId}");
        }
    }),
    Task.Run(() =>
    {
        int cant = new Random().Next(3, 10);
        for (int i = 0; i < cant; i++)
        {
            int id = Interlocked.Increment(ref idCounter);
            string tipo = tipos[new Random().Next(tipos.Length)];
            var v = new Vehiculo(id, tipo, "Sur");
            lock (listaVehiculos) { listaVehiculos.Add(v); }
            Console.WriteLine($"[Sur] Vehiculo {v.Id} ({v.Tipo}) generado - Task ID: {Task.CurrentId} - Hilo: {Thread.CurrentThread.ManagedThreadId}");
        }
    }),
    Task.Run(() =>
    {
        int cant = new Random().Next(3, 10);
        for (int i = 0; i < cant; i++)
        {
            int id = Interlocked.Increment(ref idCounter);
            string tipo = tipos[new Random().Next(tipos.Length)];
            var v = new Vehiculo(id, tipo, "Centro");
            lock (listaVehiculos) { listaVehiculos.Add(v); }
            Console.WriteLine($"[Centro] Vehiculo {v.Id} ({v.Tipo}) generado - Task ID: {Task.CurrentId} - Hilo: {Thread.CurrentThread.ManagedThreadId}");
        }
    })
};

amait Task.WhenAll(tareasGeneracion);
Console.WriteLine($"\nTotal vehiculos generados: {listaVehiculos.Count}\n");
```

3 Zonas con semáforos independientes

Se crearon 3 zonas — Norte, Sur y Centro — cada una con su propio semáforo, calle e intersección. Cada semáforo tiene tiempos distintos para simular comportamiento real de tráfico urbano.

```
// 3 zonas con su propio semáforo, calle e intersección
var semaforoNorte = new Semaforo(id: 1, tiempoVerde: 3000, tiempoAmarillo: 1000, tiempoRojo: 3000);
var semaforoSur = new Semaforo(id: 2, tiempoVerde: 2000, tiempoAmarillo: 1000, tiempoRojo: 4000);
var semaforoCentro = new Semaforo(id: 3, tiempoVerde: 4000, tiempoAmarillo: 1000, tiempoRojo: 2000);

var calleNorte = new Calle(1, "Zona Norte", capacidadMaxima: 3);
var calleSur = new Calle(2, "Zona Sur", capacidadMaxima: 3);
var calleCentro = new Calle(3, "Zona Centro", capacidadMaxima: 3);

var interseccionNorte = new Interseccion(1, (5, 5), semaforoNorte, calleNorte, calleNorte);
var interseccionSur = new Interseccion(2, (10, 10), semaforoSur, calleSur, calleSur);
var interseccionCentro = new Interseccion(3, (15, 15), semaforoCentro, calleCentro, calleCentro);

var zonaInfraestructura = new Dictionary<string, (Semaforo semaforo, Calle calle, Interseccion interseccion)>

{ "Norte",  (semaforoNorte,  calleNorte,  interseccionNorte)  },
{ "Sur",    (semaforoSur,    calleSur,    interseccionSur)    },
{ "Centro", (semaforoCentro, calleCentro, interseccionCentro) }
```

Nuevo tipo de vehículo: Camion

Se agregó el tipo Camion con velocidad base entre 1 y 2

```
int probabilidadColision = Tipo switch
{
    "Moto" => 20, // mas riesgo
    "Auto" => 10,
    "Bus" => 5, // mas seguro
    "Camion" => 5,
    _ => 10
};
```

Teclas B, P, V para agregar vehículos por zona

Se agregaron 3 nuevas teclas al controlador de teclado para que el usuario pueda agregar vehículos en una zona específica durante la simulación.

```
switch (keyInfo.Key)
{
    case ConsoleKey.A:
        AgregarVehiculo();
        break;
    case ConsoleKey.F:
        esperandoId = true;
        _idBuffer = "";
        Console.Write("\n[Teclado] ID del vehículo (Enter para confirmar): ");
        break;
    case ConsoleKey.Q:
        Terminar();
        return;
    case ConsoleKey.B:
        AgregarVehiculoEnZona("Sur");
        break;
    case ConsoleKey.P:
        AgregarVehiculoEnZona("Norte");
        break;
    case ConsoleKey.V:
        AgregarVehiculoEnZona("Centro");
        break;
}
```

```
private void AgregarVehiculoEnZona(string zona)
{
    var tipos = new[] { "Auto", "Bus", "Moto", "Camion" };
    int id = Interlocked.Increment(ref _nextId);
    string tipo = tipos[new Random().Next(tipos.Length)];
    var nuevo = new Vehiculo(id, tipo, zona);
    lock (_listaVehiculos) { _listaVehiculos.Add(nuevo); }
    Console.WriteLine($"\n[Teclado] Vehículo {id} ({tipo}) agregado en Zona {zona}.");
}
```